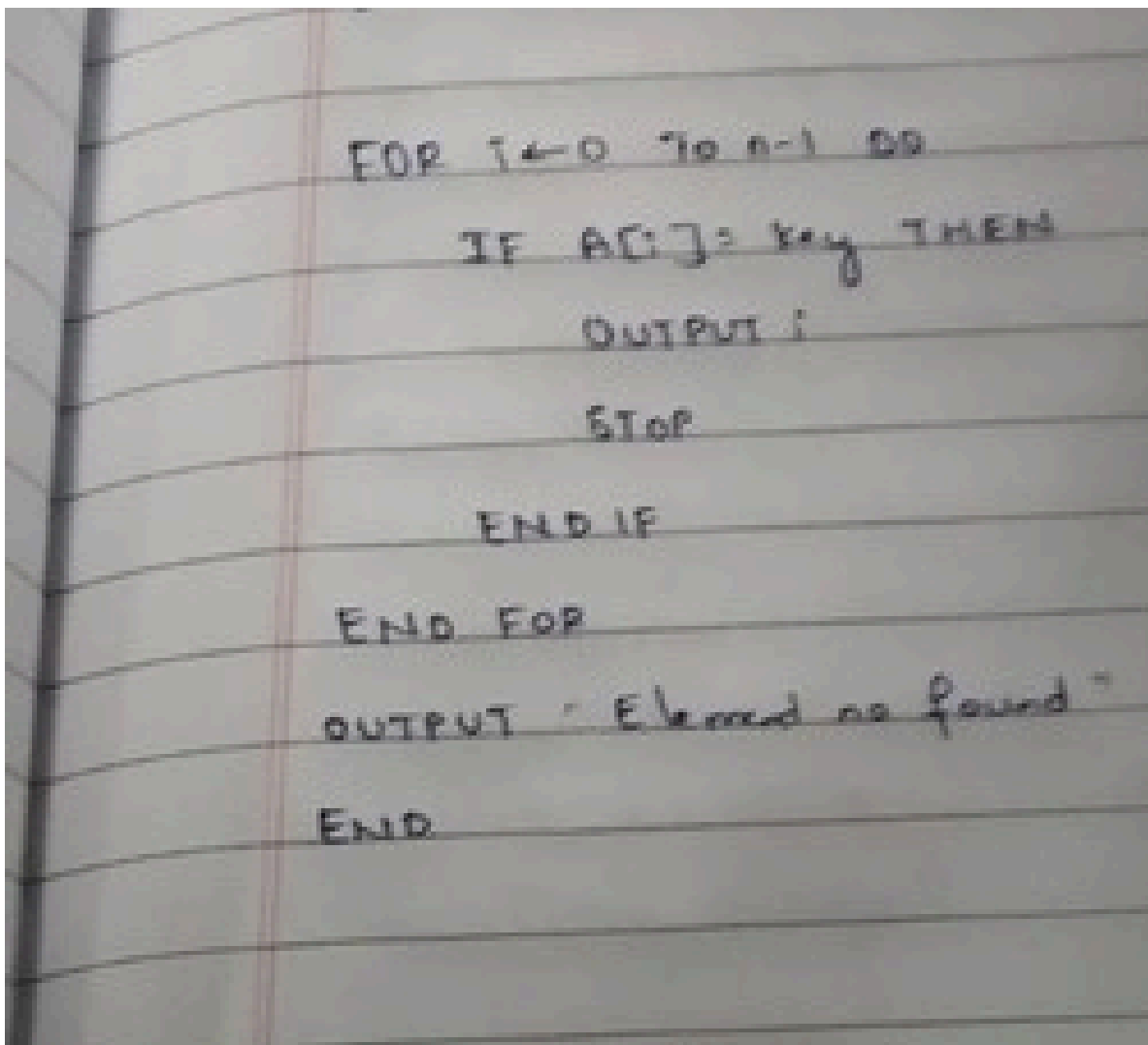


Practical-2

Aim:

Implementation and Time analysis of linear and binary search algorithm.

1) Linear search



```
FOR I ← 0 TO n-1 DO
    IF A[I] = key THEN
        OUTPUT I
        STOP
    END IF
END FOR
OUTPUT "Element not found"
END
```

The image shows a handwritten pseudocode for a linear search algorithm on lined paper. The code is written in a clear, legible hand. It starts with a 'FOR' loop from 0 to n-1. Inside the loop, there is an 'IF' statement checking if the element at index I is equal to the 'key'. If true, it prints the index I and stops the program. If the loop completes without finding the key, it prints a message indicating the element was not found and then ends the program.

code:

```
#include <iostream>
using namespace std;

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }

    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);

    int key = 30;

    int result = linearSearch(arr, n, key);

    if (result != -1)
        cout << "Element found at index " << result;
    else
        cout << "Element not found";

    return 0;
}
```

Output:

```
Output
Element found at index 2
=== Code Execution Successful ===
```

Time complexity:

Time complexity:-

1. Best case:-
The element is present at the first position.
Ex:- A = 10 20 30 40 50
key = 10
 $T(n) = 1$
 $T(n) = O(1)$

2. Average case:-
Suppose the element is somewhere in the middle
Ex:- A = 10 20 30 40 50
key = 30
 $T(n) = \frac{n}{2}$
 $T(n) = O(n)$

3. Worst case:-
Element in last position or not element present
A = 10 20 30 40 50
key = 50
 $T(n) = n$
 $T(n) = O(n)$

1) Binary search

```
WHILE low ≤ high DO  
    mid ← (low + high) / 2  
    IF A[mid] = key THEN  
        OUTPUT mid  
        STOP  
    ELSE IF A[mid] < key THEN  
        low ← mid + 1  
    ELSE  
        high ← mid - 1  
    END IF  
END WHILE  
OUTPUT "Element not found"  
END
```

code:

```
#include <iostream>
using namespace std;

int binarySearch(int arr[], int n, int key) {
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key) {
            return mid;
        }
        else if (arr[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }

    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);

    int key = 40;

    int result = binarySearch(arr, n, key);

    if (result != -1)
        cout << "Element found at index " << result;
    else
        cout << "Element not found";

    return 0;
}
```

Output:

Output

Element found at index 3

=== Code Execution Successful ===

Time complexity:

1) Best case:-

The element is found at the middle position in the first comparison.

Ex: $A = 10 \ 20 \ 30 \ 40 \ 50$

key = 30

$A[\text{mid}] = \text{key}$

$$T(n) = 1$$

$$T(n) = O(1)$$

2) Average case:-

On average, the search also eliminates approximately half of the remaining elements at each step.

$$T(n) \approx \log_2 n$$

$$T(n) = O(\log n)$$

3) Worst case

This is where Binary Search becomes interesting.

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log_2 n$$

$$T(n) = \log n$$

$$T(n) = O(\log n)$$

Conclusion:

In this practical, we implemented Linear Search and Binary Search and understood their working and time complexities. Linear Search checks elements one by one, while Binary Search repeatedly divides a sorted array into halves. Thus, Binary Search is more efficient for large sorted datasets.